

INSERT: cómo cargar datos en tus tablas

Las tablas de Tienda Latam están creadas, con sus columnas, tipos de datos, restricciones y relaciones. Hasta ahora son contenedores vacíos. **INSERT** es el comando que empieza a llenarlos con información real.

Hay más de una forma de usarlo, y elegir la correcta hace una diferencia concreta: insertar cien filas una por una puede tardar minutos; con la forma adecuada, tarda segundos.

INSERT básico: fila por fila con columnas explícitas

La forma más directa es nombrar las columnas en las que vas a insertar y luego dar los valores correspondientes.

```
SQL
INSERT INTO paises (nombre, codigo)
VALUES ('Chile', 'CHL');

INSERT INTO paises (nombre, codigo)
VALUES ('Argentina', 'ARG');
```

La estructura es siempre la misma: **INSERT INTO**, el nombre de la tabla, las columnas entre paréntesis, **VALUES** y los datos en el mismo orden.

Nombrar las columnas explícitamente es una buena práctica porque hace el código más claro y protege el INSERT si alguien agrega o reordena columnas en la tabla en el futuro. Si no nombras las columnas, el orden de los valores tiene que coincidir exactamente con el orden en que se definieron, lo que puede generar errores difíciles de detectar.

Este formato funciona bien para pocos registros. Cuando los datos son muchos, hay una forma mejor.

INSERT múltiple: varios registros en una sola instrucción

En lugar de repetir `INSERT INTO` para cada fila, puedes incluir todos los registros en un solo `VALUES`, separados por comas.

```
SQL
INSERT INTO paises (nombre, codigo)
VALUES
    ('Colombia', 'COL'),
    ('México', 'MEX'),
    ('Perú', 'PER'),
    ('Brasil', 'BRA');
```

El resultado es el mismo que hacer cuatro `INSERT` separados, pero en una sola ejecución. Más eficiente, más limpio y mucho más fácil de revisar cuando tienes muchos registros.

RETURNING: verificar lo que acabas de insertar

Normalmente, para confirmar que un `INSERT` funcionó hay que hacer un `SELECT` por separado. `RETURNING` te evita ese paso: le pides al motor que te devuelva los datos del registro que acaba de insertar, justo en el momento en que lo hace.

Devolver columnas específicas:

```
SQL
INSERT INTO categorias (nombre)
VALUES ('Electrónica')
RETURNING categoria_id, nombre;
```

El motor inserta el registro y devuelve inmediatamente el ID generado y el nombre. Útil para confirmar que la llave primaria se asignó correctamente.

Devolver todo:

```
SQL
INSERT INTO tipo_cliente (nombre)

VALUES ('Premium')

RETURNING *;
```

El asterisco devuelve todas las columnas del registro insertado. Práctico cuando quieres ver el estado completo de la fila tal como quedó guardada, incluyendo los valores que el motor asignó por defecto.

RETURNING también funciona con INSERT múltiple:

```
SQL
INSERT INTO tipo_cliente (nombre)

VALUES

    ('Estándar'),

    ('Mayorista'),

    ('Corporativo')

RETURNING tipo_cliente_id, nombre;
```

Devuelve una fila por cada registro insertado, con los IDs generados automáticamente.

INSERT con llave foránea: el orden importa

Cuando una tabla tiene llaves foráneas, los valores que insertas deben existir previamente en las tablas referenciadas. Si no existen, el motor rechaza el INSERT.

Por ejemplo, para insertar un cliente necesitas que el `tipo_cliente_id` que vas a usar ya esté creado en la tabla `tipo_cliente`:

```
SQL
-- Primero esto tiene que existir:

-- tipo_cliente_id = 1 en la tabla tipo_cliente

INSERT INTO clientes (nombre, correo, tipo_cliente_id)
VALUES ('María González', 'maria@email.com', 1)
RETURNING cliente_id, nombre, correo;
```

Si intentas insertar con un `tipo_cliente_id` que no existe en la tabla padre, el motor devuelve un error de integridad referencial. Eso es exactamente lo que quieres: que los datos incorrectos no entren.

El mismo principio aplica para cualquier tabla con llaves foráneas. El orden lógico de carga es el mismo orden en que creaste las tablas: primero las tablas padre, luego las hijas.

Un INSERT múltiple real: la tabla de clientes

Aquí un ejemplo con varios clientes a la vez, respetando la llave foránea de `tipo_cliente_id`:

```
SQL
INSERT INTO clientes (nombre, correo, activo,
tipo_cliente_id)
VALUES
    ('María González', 'maria@email.com', TRUE, 1),
    ('Ana Martínez', 'ana@email.com', TRUE, 1),
    ('Carlos López', 'carlos@email.com', TRUE, 2)
RETURNING cliente_id, nombre, correo;
```

Los tres clientes se insertan en una sola operación. **RETURNING** confirma que los IDs se generaron y que los datos quedaron bien.

Referencia rápida

Forma de INSERT	Cuándo usarla
INSERT básico	Pocos registros, necesitas ser explícito
INSERT múltiple	Muchos registros de una sola vez
INSERT con RETURNING	Quieres confirmar el resultado sin hacer un SELECT
INSERT con llave foránea	Siempre que la tabla tenga relaciones definidas

Con los datos cargados, el siguiente paso es consultarlos: filtrarlos, ordenarlos y extraer exactamente la información que necesitas. Para eso existe **SELECT**, y es donde SQL empieza a mostrar todo su valor como herramienta de análisis.